
PROJECT III - Game Programming Projects

# DUAL REALMS

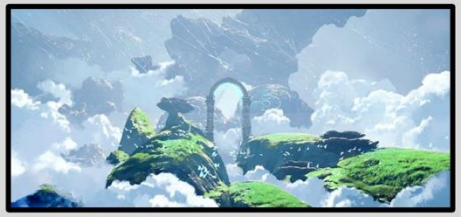
A 2D Platform Jumping Game

# DUAL REALMS: A QUEST FOR TREASURE

I have always been fond of side-scrolling platform games, so I once attempted to design a level inspired by them. Before starting, I collected a lot of reference materials from different games to gain a general understanding of their design concepts.adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quis ipsum suspendisse ultrices gravida. Risus commodo viverra maecenas accumsan lacus vel facilisis.


**SOME REFERENCE**

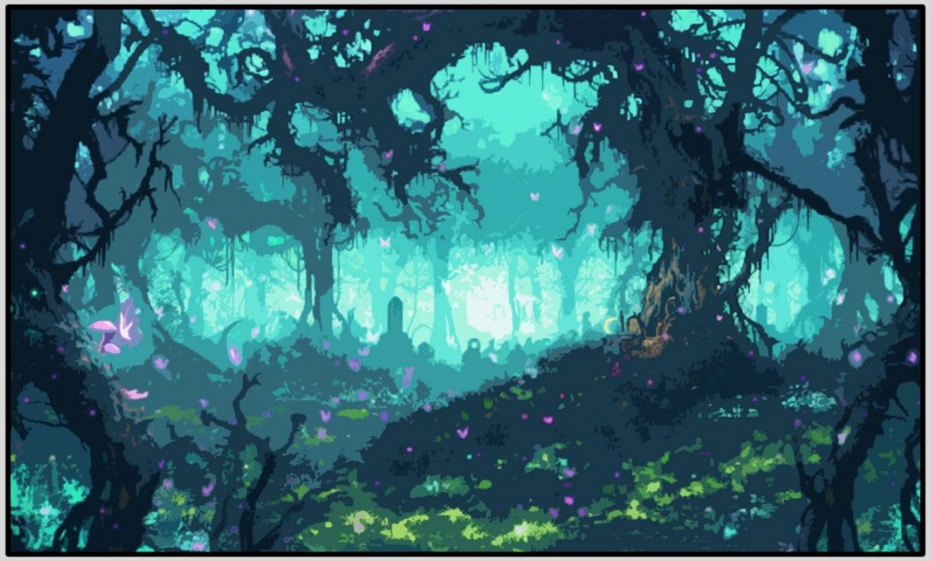

# MOODBOARD



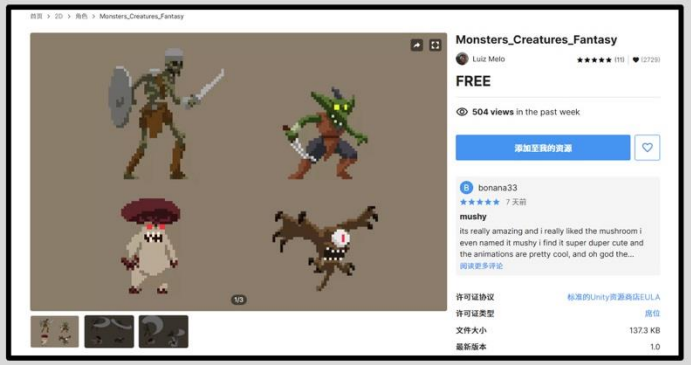
After studying various examples of side-scrolling games, I began designing my own game. The worldbuilding and setting were kept relatively simple, as overly complex designs could lead to significantly more programming challenges. The story revolved around a hero who accidentally enters an illusionary realm in search of treasure. At the time, I planned to create two levels and gathered some artistic references, designing unique background illustrations for each level.

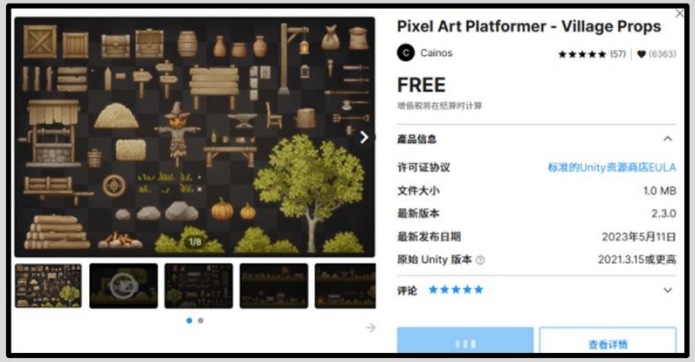
# THUMBNAILS

## STAGE 1

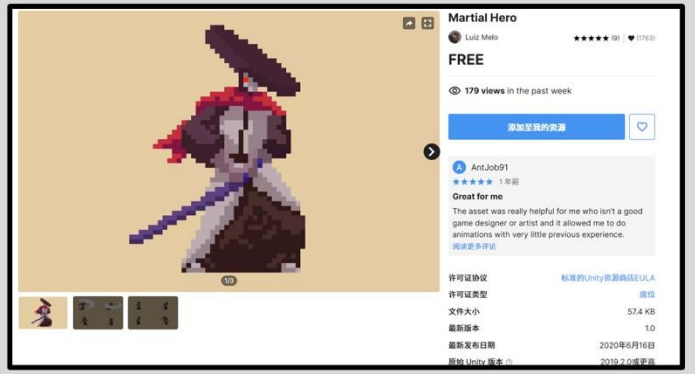


I finalized the background designs and overall atmosphere for the two levels. The art assets for the monsters and the protagonist were sourced from the internet. Specifically, I obtained these assets from the Unity Asset Store. Below are the websites where the materials were acquired.



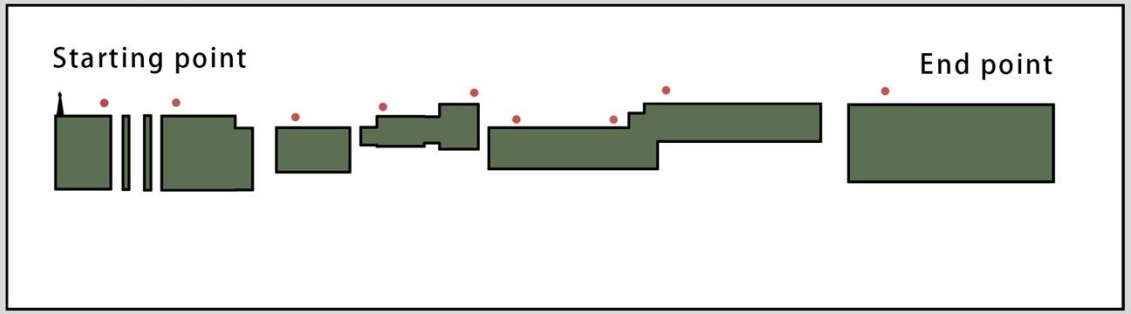

## STAGE 2



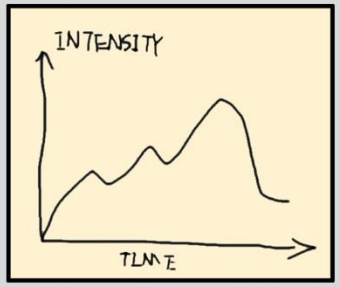

https://assetstore.unity.com/packages/2d/characters/martial-hero-170422#content
https://assetstore.unity.com/packages/2d/characters/monsters-creatures-fantasy-167949
https://assetstore-fallback.unity.com/packages/2d/environments/pixel-art-platformer-village-props-166114

Next came the crucial part: level design. A classic example of a side-scrolling platformer is Super Mario. As we all know, its levels are not particularly easy to complete. Inspired by this, I planned to make the progression routes in my game as challenging and intricate as possible, ensuring they provide a sense of accomplishment.

## VERSION 1

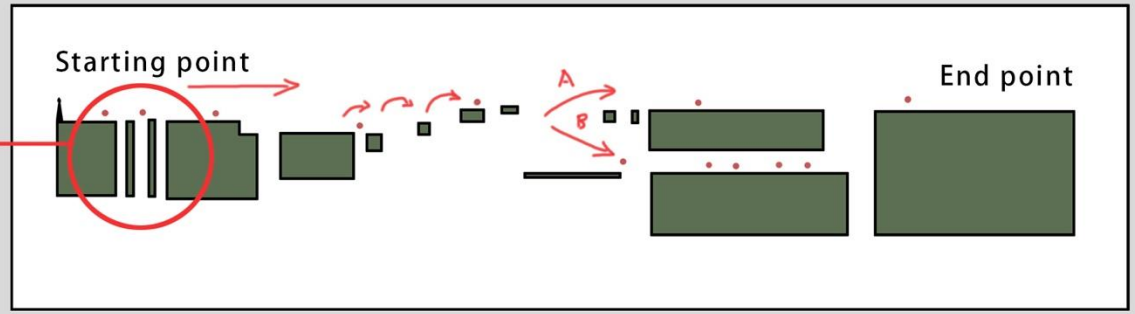



Thinking about level design rhythm




## VERSION 2

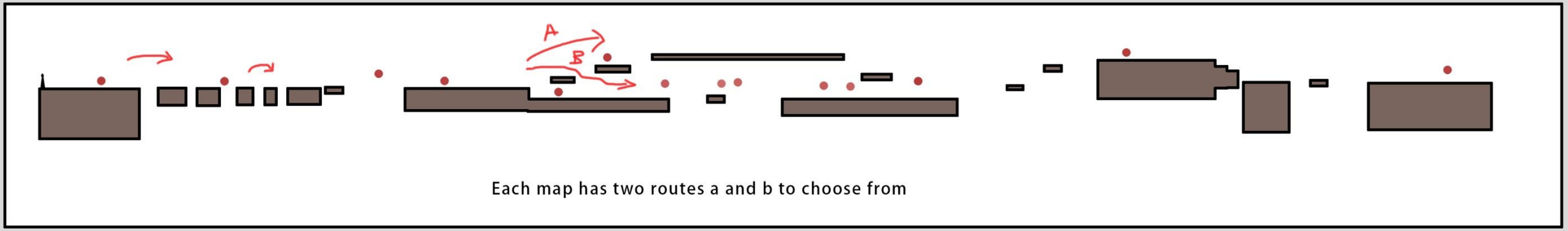
### STAGE 1




The two small platforms ahead are designed to serve as a tutorial for jumping.

Enemies

### STAGE 2




Each map has two routes a and b to choose from

## LEVEL DESIGN APPROACH:

**Difficulty Progression**: The difficulty of levels should increase in a spiral pattern, with each segment gradually progressing from easy to challenging, maintaining a sense of challenge for the player.

**Character Movement Attributes**: Before designing the levels, it's essential to define the character's movement abilities, such as jump height and distance, ensuring that the level design aligns with the character's capabilities.

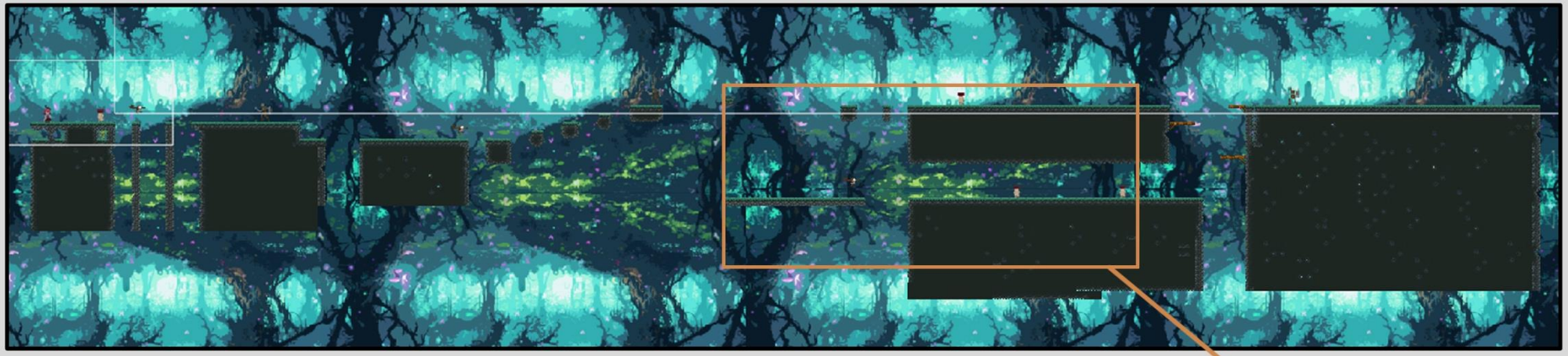
**Enemy Placement**: Enemy positions and quantities should be arranged strategically based on their attributes and the player's interaction experience, enhancing the game's strategic depth.

**Reward Distribution**: Following the principle of "reward after effort," players should receive rewards after overcoming challenges to amplify their sense of accomplishment.

# STAGE 1

I incorporated platforming mechanics alongside combat, allowing players to perform double jumps to navigate terrain and stomp on enemies to achieve victory. However, I decided not to include these mechanics in the game's help menu. Instead, I want players to experiment and discover these features on their own, as I believe this approach makes the gameplay more engaging and enjoyable.

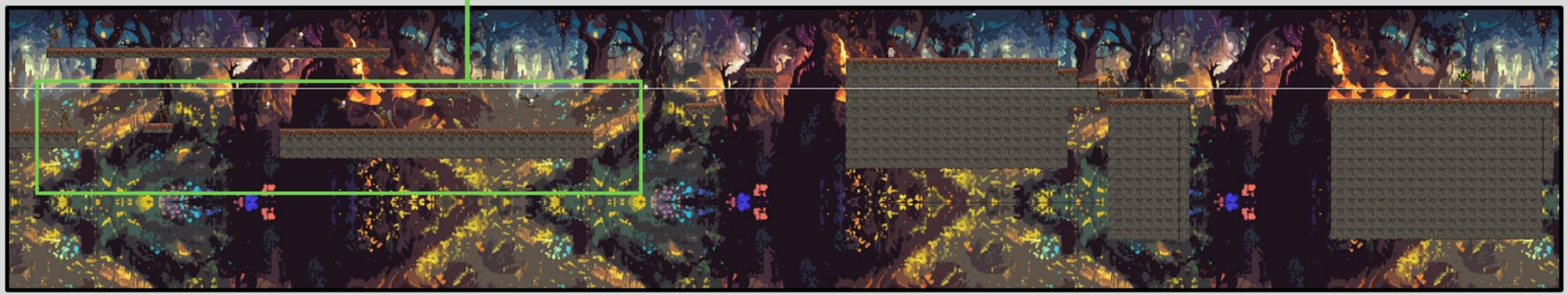
The route in the second level is designed to be more challenging, with an increased number of enemies. A boss is placed at the end of the level for players to face. Additionally, there are more sections in the level that require the use of double jumps to progress.



There are two routes here, and it's clear that the upper route is much easier. If the player accidentally falls, they must face several enemies and navigate a relatively challenging platforming section to get back on track.

# STAGE 2

The two routes in the second map are also the lower routes that are more difficult

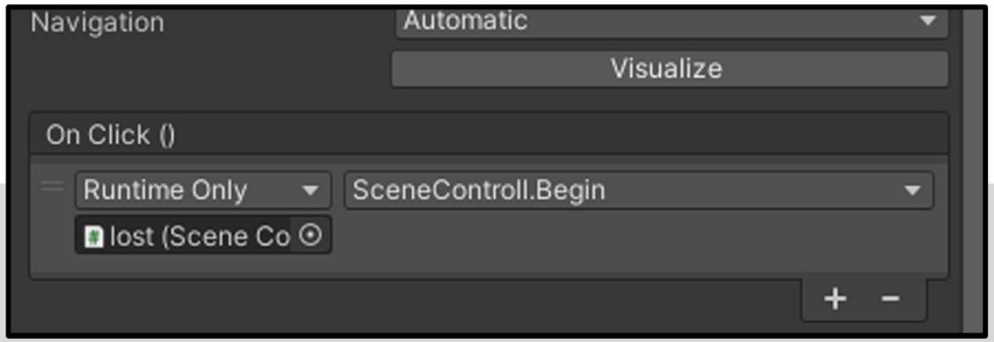


# CODE SHOWCASE

Set up the interface, hide the corresponding elements, and use code to call them as needed. Add navigation events to the buttons for functionality.

```
if (collision.gameObject.tag == "deadline")
{
    lost.active = true;
    Time.timeScale = 0;
}
//胜利
if (collision.gameObject.tag == "win")
{
    win.active = true;
}
```

```
void lost1()
{
    lost.active = true;
    Time.timeScale = 0;
}
```




Write character behavior code to handle action switching and trigger the destruction code for enemies when they are defeated. Use events in the timeline to ensure enemies disappear only after the explosion animation finishes playing. (The character code diagram is shown here; additional details are in the attachment.)

The character movement code uses the velocity property of the Rigidbody component to give the player momentum for moving left or right, enabling horizontal movement. The character's facing direction is controlled based on the movement direction. Jumping is implemented using the "K" key, with a limit set to allow only two jumps.

```
private void OnCollisionEnter2D(Collision2D collision)
{
    //是否碰到标签为enemy的物体
    if (collision.gameObject.tag == "enemy"|| collision.gameObject.tag == "piranha")
    {
        //如果处于fall状态并且在它上方就踩死它
        if (anim.GetBool("fall")&&(this.transform.position.y-0.6f> collision.gameObject.transform.position.y))
        {
            //判断哪个类型怪物调用代码
            if(collision.gameObject.tag == "enemy")
            { collision.gameObject.GetComponent<bee>().dead(); }
            else
            { collision.gameObject.GetComponent<piranha>().dead(); }
            Rb.velocity = new Vector2(Rb.velocity.x, 10);
            anim.SetBool("jump", true);
            anim.SetBool("idle", false);
            anim.SetBool("run", false);
            extrajump = 1;
        }
        //不然的话就受伤并且在怪物的那一侧就往那一次后退
        else if (transform.position.x < collision.gameObject.transform.position.x)
        {
            ismove = false;
            anim.SetBool("hurt",true);
            Rb.velocity = new Vector2(-5,5);

        }
        else if (transform.position.x > collision.gameObject.transform.position.x)
        {
            ismove = false;
            anim.SetBool("hurt", true);
            Rb.velocity = new Vector2(5,5);

        }
    }
    if (collision.gameObject.tag == "deadline")
    {
        lost.active = true;
        Time.timeScale = 0;
    }
    //胜利
    if (collision.gameObject.tag == "win")
    {
        win.active = true;
    }
}
```

```
void newjump()
{
    if (Isground)
    {
        extrajump = 2;
    }
    if (Input.GetButtonDown("Jump") && extrajump > 0)
    {
        jumpaudio.Play();
        Rb.velocity = Vector2.up * jumpforce;
        extrajump--;
        anim.SetBool("jumping", true);

    }
    if (Input.GetButtonDown("Jump") && extrajump == 0 && Isground)
    {
        jumpaudio.Play();
        Rb.velocity = Vector2.up * jumpforce;

        anim.SetBool("jumping", true);

    }
}
```

The collision code implements different effects based on the type of object collided with. If the character collides with an enemy, it first checks whether the character is in a falling state. If so, and the height difference exceeds a certain threshold, the enemy is defeated; otherwise, a damage effect is played. If the character collides with a death line, the game ends. If the collision is with the house collider, the game displays a victory screen.

```
void swithanim()
{
    //跳跃速度小于0切换下落动画
    if (Rb.velocity.y < 0)
    {
        anim.SetBool("idle", false);
        anim.SetBool("jump", false);
        anim.SetBool("fall", true);
    }
    //接触到ground图层就切换回idle状态
    if (circoll.IsTouchingLayers(ground) && anim.GetBool("fall"))
    {
        anim.SetBool("fall", false);
        anim.SetBool("idle", true);
        extrajump = 2;
    }
}
```

# HERO & ENEMYENEMY MECHANICS SHOWCASE

Moving Platform: Define two points on the platform, allowing it to move between them. After obtaining the coordinates of the two points, delete the points to prevent them from interfering with the platform's movement.

```
{
    //获取组件
    rb = GetComponent<Rigidbody2D>();
    anim = GetComponent<Animator>();
    leftpoint = left.position.x;
    rightpoint = right.position.x;
    Destroy(left.gameObject);
    Destroy(right.gameObject);
    coll = GetComponent<Collider2D>();
}

void Update()
{
    move();
}
void move()
{
    //怪物掉头以及改变速度方向
    if ((this.transform.position.x < leftpoint || this.transform.position.x > rightpoint) && change)
    {
        speed = speed * -1;
        change = false;
    }
    //防止来回掉头的情况
    if ((this.transform.position.x > leftpoint && this.transform.position.x < rightpoint))
    {
        change = true;
    }
    //保持原有速度
    rb.velocity = new Vector2(speed * Time.fixedDeltaTime, rb.velocity.y);

}
```

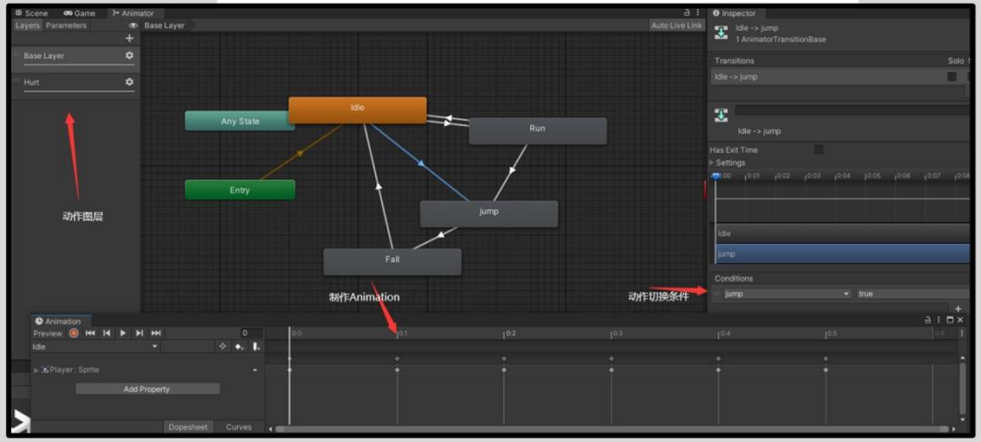
Configure the protagonist's prefab and add each of the character's actions to the Animator, organizing them by action names. After setting up the idle, jump, fall, run, and hurt actions, open the Animation window to define these actions. Establish the transitions between actions and specify the transition durations.

```
{
    //获取组件
    rb = GetComponent<Rigidbody2D>();
    anim = GetComponent<Animator>();
    leftpoint = left.position.x;
    rightpoint = right.position.x;
    Destroy(left.gameObject);
    Destroy(right.gameObject);
    coll = GetComponent<Collider2D>();
}

void Update()
{
    move();
}
void move()
{
    //怪物掉头以及改变速度方向
    if((this.transform.position.x<leftpoint|| this.transform.position.x > rightpoint )&& change)
    {
        this.transform.localScale = new Vector3(this.transform.localScale.x * -1, 1, 1);
        speed = speed * -1;
        change = false;
    }
    //防止来回掉头的情况
    if ((this.transform.position.x > leftpoint && this.transform.position.x < rightpoint))
    {
        change = true;
    }
    //保持原有速度
    rb.velocity = new Vector2(speed * Time.fixedDeltaTime, rb.velocity.y);

}
```

For Enemy One and Enemy Two, place two points on either side to allow them to move between the points. When reaching the left point, they will change direction.




```
{
    //获取组件，找到游戏中名为player的物体
    rb = GetComponent<Rigidbody2D>();
    anim = GetComponent<Animator>();
    coll = GetComponent<Collider2D>();
    player = GameObject.Find("Player").GetComponent<Transform>();
}
void Update()
{
    //判断是否进入攻击范围，并且是否到攻击间隔
    if(Mathf.Abs(player.position.x-transform.position.x)<1.8f&&attacktime>Maxtime)
    {
        //是时攻击
        anim.SetTrigger("attack");
        attacktime = 0;
    }

    attacktime += Time.deltaTime;
}

public void attack()
{
    //再次判断是否还在攻击范围
    if (Mathf.Abs(player.position.x - transform.position.x) < 1.8f&& Mathf.Abs(player.position.y - transform.position.y)<1.5f)
    {
        player.gameObject.GetComponent<PlayerControll>().piranhaattack();
    }
}
public void dead()
```

In the code for Enemy Three, retrieve the character's position. When the character enters the flower's attack range, switch to the attack animation and trigger the character's damage effect.

```
public void Hurt()
{
    //减少生命
    for(int i=0;i<10;i++)
    {
        life -= 1;
    }
    //切换动画
    anim.SetBool("hurt", false);
    ismove = true;
}
```

# SCREENSHOTS OF THE GAME DEMO

Main interface


Kill different enemies


Kill different enemies


Jump kill


Get the treasure